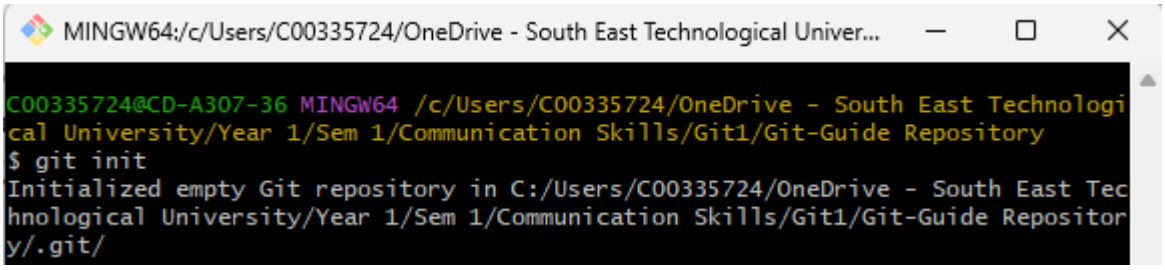
Git1 – Joe Cummins

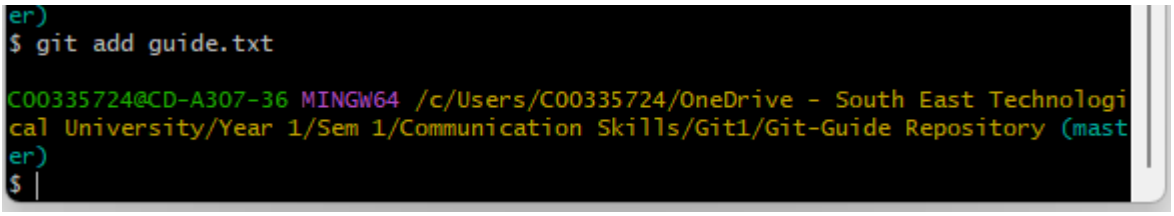
init

‘git init’ creates a brand new local Git repository inside a file folder so Git can start tracking it. It adds a hidden .git folder that tracks the history of the changes made. In the command window you would type **git init** , an example is shown;



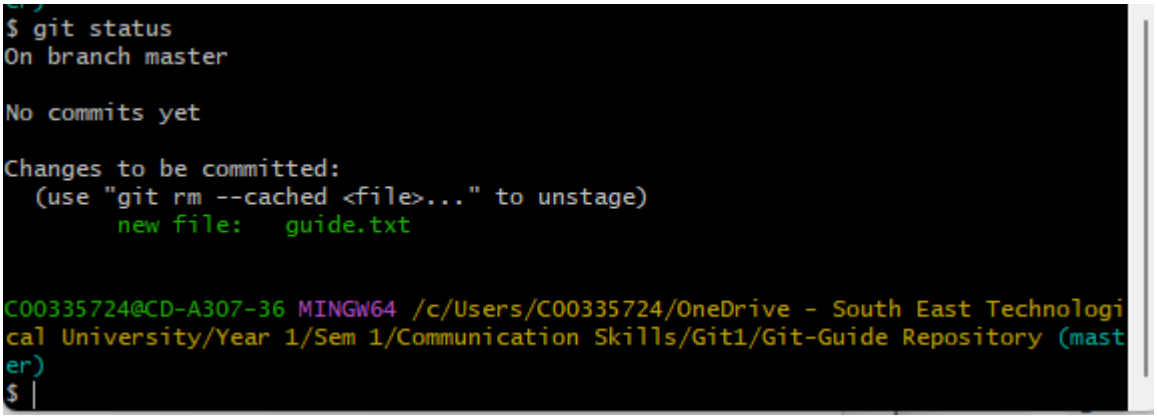
add

‘git add’ tells Git which file changes should be included in the next save point. Git add should be used right before a commit. Basically, it moves them to the staging area so you can commit without errors. In the command window you would type **git add .** or **git add file.fileExtension** , an example is shown here;



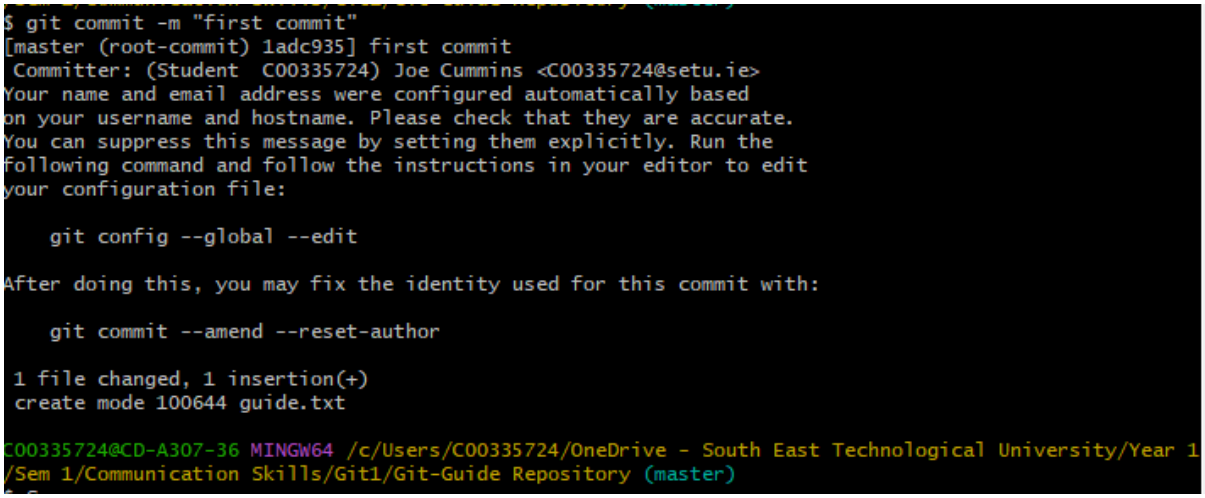
status

‘git status’ shows a list of untracked files in red, and staged files in green. The purpose of this is to see exactly which files you have changed. In the command window you would type **git status** , an example is shown here;



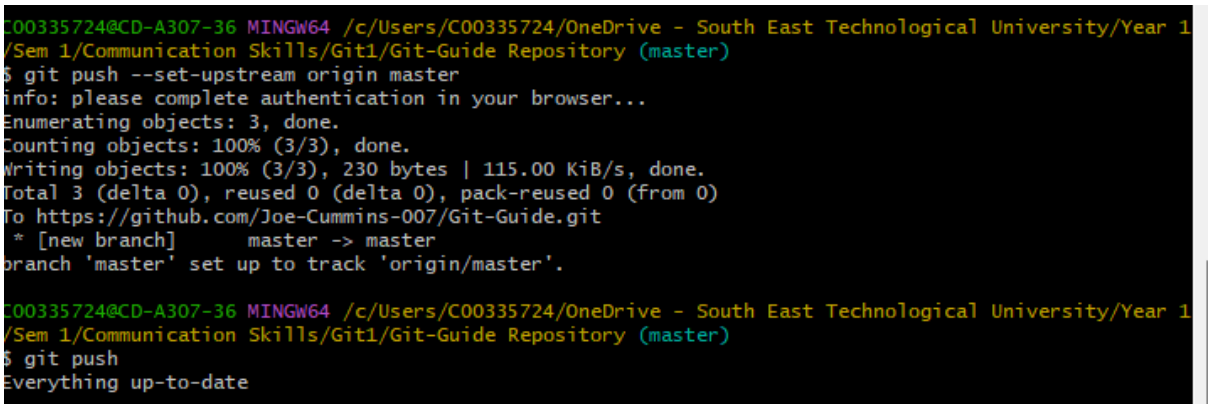
commit

‘git commit’ creates a permanent save of the files you have edited on the local repository only. This should be used to save a small piece of working code. In the command window you should type **git commit -m “message”** , an example is shown here;



push

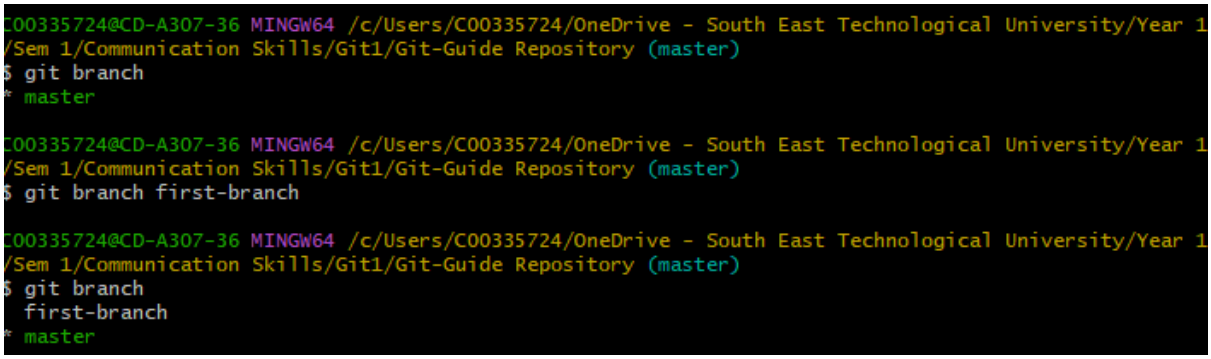
‘git push’ uploads all of the files from your local repository into a remote repository, this would typically be used in group work for others to see your work. In the command window you would type **git push –set-upstream origin master** for the very first push only, and **git push** for every save after that , an example is shown here;



branch

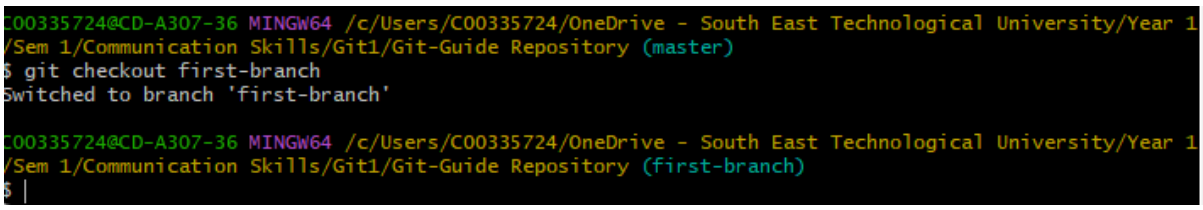
‘git branch’ can create, list, rename and delete branches in the repository. This would be used to add a new variation of something that is already done, for eg adding a feature on your on copy of the repository. In the command window you would type **git branch** to list the branches, and **git branch name-name** , which would create a branch called

name-name. An example is shown here;



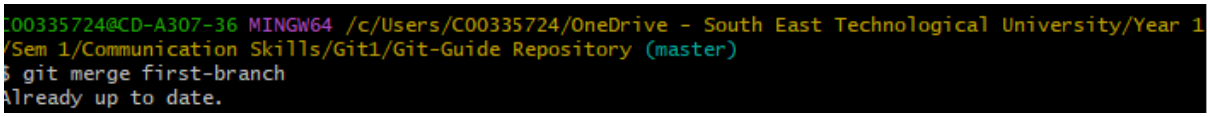
checkout

'git checkout' lets you move between different branches, which allows you to access different versions of your code, etc. In the command window you would type **git checkout file.fileExtension** , an example is shown here;



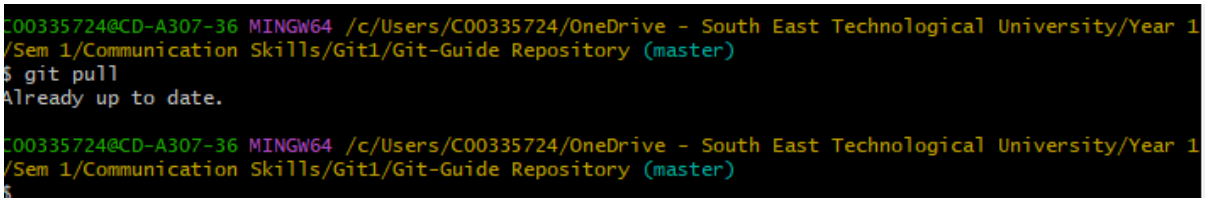
merge

'git merge' combines two branches into a single branch. This would be used to take code from two different branches and make them into on bigger or main branch. In the command window you would type **git merge name-name** , which will merge name-name into the branch you are currently in. An example is shown here;



pull

'git pull' downloads changes from a remote repository and merges them into you local branch. This would be used to download files from a member in your group before merging them into your own. In the command window you would type **git pull** , and anyone with access to the repository with changes will be pulled to your local branch. An example is shown here, although I have nothing to pull;



switch

‘git switch’ changes your workspace to a different branch timeline, or to the same one. This can be used to edit old files made days, weeks or months ago, as long as the save file is there. In the command window you would type **git switch main** to jump to the main branch and **git switch -c feature-api** to create feature-api and switch to it. An example is shown here;

